

Graph Mining: Methods for Mining Frequent Subgraphs

Graph mining is a crucial area of data mining focused on discovering useful patterns and relationships in **graph-structured data**. Real-world data such as **social networks**, **chemical compounds**, **protein interactions**, and **transportation networks** can be represented as graphs, where:

- **Nodes (V)** represent entities.
- **Edges (E)** represent relationships among them.

Formally, a graph can be represented as:

where λ denotes the labeling function that assigns labels to vertices or edges.

The goal of **frequent subgraph mining (FSM)** is to identify **subgraphs** that occur frequently across a set of graphs (graph database).

Example: In a collection of chemical compound graphs, frequent subgraph mining helps identify common molecular structures (e.g., benzene rings).

Definition of Frequent Subgraph

Let:

- G be a database of graphs.
- H be a subgraph pattern.

The **support** of subgraph H in G is defined as:

A subgraph is **frequent** if:

where min_sup is the user-defined minimum support threshold.

Methods for Mining Frequent Subgraphs

Frequent Subgraph Mining (FSM) is one of the core tasks in **graph mining**, aimed at discovering all subgraphs that appear frequently in a set of graphs.

Given a graph database, the objective is to find all subgraphs such that:

Here, min_sup is a user-defined threshold indicating the minimum acceptable frequency of occurrence.

Mining frequent subgraphs is more complex than traditional itemset mining due to **graph isomorphism**, **non-sequential structure**, and **exponential search space**.

Therefore, efficient algorithms are required to traverse, represent, and prune the search space effectively.

Broadly, FSM methods are categorized into the following main approaches:

1. Apriori-Based Methods

The **Apriori-based approach** extends the idea of the *Apriori principle* from itemset mining to graph mining.

The principle states:

“If a graph pattern is frequent, then all of its subgraphs must also be frequent.”

Thus, the search process follows a **bottom-up, breadth-first** strategy, starting from small frequent subgraphs and progressively joining them to form larger ones.

1.1 Working Mechanism

1. **Initialization:**

Identify all frequent **1-edge subgraphs** by scanning the database and computing their support.

2. **Candidate Generation:**

Generate candidate subgraphs of size $k+1$ by combining two frequent subgraphs of size k that share a common substructure.

This operation is known as **join operation**.

3. **Frequency Counting:**

For each candidate subgraph, test whether it occurs in each graph of the database using **subgraph isomorphism** (a computationally expensive matching process).

4. **Pruning:**
Eliminate candidate subgraphs that do not satisfy the minimum support threshold.
5. **Iteration:**
Repeat the above steps by increasing subgraph size until no new frequent subgraphs are found.

1.2 Key Algorithms

- **AGM (Apriori-based Graph Mining):**
One of the earliest FSM algorithms. It extends the Apriori algorithm by enumerating subgraphs using adjacency matrices and performing level-wise joins.
- **FSG (Frequent Subgraph Discovery):**
Introduces a more efficient candidate generation method using **DFS traversal** and canonical labeling to avoid duplicate candidates.

1.3 Advantages

- Straightforward concept derived from Apriori association rule mining.
- Ensures completeness — all frequent subgraphs are found.

1.4 Limitations

- **High computational complexity** due to a massive number of candidate subgraphs.
- **Repeated isomorphism testing**, which is NP-complete.
- Not scalable for large or dense graph datasets.

2. Pattern-Growth Methods

The **pattern-growth** strategy addresses the inefficiencies of candidate generation in Apriori-based methods.

Instead of generating a large number of candidates, this approach **grows frequent subgraphs directly** by recursively extending smaller frequent ones.

It employs **depth-first search (DFS)** to explore the search space efficiently.

2.1 Basic Idea

If a subgraph is frequent, then its **extensions** (created by adding an edge or vertex) are tested for frequency.

This can be formally represented as:

where e is a new edge connecting vertices u and v , and l is the edge label.

Each extension is verified for frequency without generating all possible combinations as in Apriori-based algorithms.

2.2 Key Algorithms

(a) gSpan (Graph-based Substructure Pattern Mining)

- Introduced by Yan and Han, gSpan is one of the most influential FSM algorithms.
- It uses **DFS codes** to represent graphs in a **canonical form**.
- Builds a **DFS code tree**, where each node corresponds to a unique subgraph.
- Subgraphs are extended by adding edges, ensuring that duplicate subgraphs are not generated.

Algorithmic Steps:

1. Assign a DFS code to each graph — this acts as a unique, minimal identifier.
2. Construct a **search tree** where each node represents a DFS code (subgraph).
3. Perform **pattern growth** by extending the current subgraph using forward or backward edge extension rules.
4. Compute the support of the new subgraph and prune those below the *min_sup* threshold.

Advantages:

- Avoids candidate generation and redundancy.
- Reduces isomorphism checking using canonical DFS codes.
- Scales better than Apriori-based algorithms.

Equation for Canonical Form:

For two DFS codes c_1 and c_2 :

where c_1 ensures a unique and minimal representation for each graph.

(b) CloseGraph

CloseGraph is an extension of gSpan designed to find **closed frequent subgraphs** — subgraphs that are not a subgraph of any other graph with the same support.

Definition:

A subgraph is closed if there does not exist any supergraph such that:

Advantages:

- Eliminates redundancy by reporting only the most specific frequent patterns.
- Reduces output size significantly.

(c) SPIN (Spanning Tree-based Subgraph Mining)

SPIN uses **spanning trees** to represent graph structures compactly and performs subgraph growth on these trees.

It reduces the number of generated candidates by leveraging spanning tree equivalence classes.

Advantages:

- Reduces computational complexity.
- Efficient for large sparse graphs.

Limitations:

- May miss certain non-tree-like structures.

3. Constraint-Based Methods

In many real-world scenarios, not all frequent patterns are interesting. To reduce the number of mined patterns, **constraints** are introduced to focus only on relevant subgraphs.

Common Constraints Include:

- **Structural Constraints:** e.g., connected subgraphs only, or tree-like subgraphs.
- **Label Constraints:** specific vertex or edge labels must be included.
- **Size Constraints:** limit on the number of nodes or edges.
- **Topological Constraints:** specific properties such as cycles, cliques, or paths.

Example:

Mining only connected subgraphs with ≤ 6 vertices and containing at least one “C–C” bond in a chemical compound dataset.

These constraints help **prune the search space** early, improving efficiency.

4. Approximate and Probabilistic Methods

When dealing with **massive, uncertain, or noisy graph data**, exact frequent subgraph mining becomes computationally infeasible.

Approximate and probabilistic methods aim to discover subgraphs that occur *frequently with high probability* rather than deterministically.

Example Algorithms:

- **GraphSig:** Uses a probabilistic hashing mechanism to summarize graphs and approximate frequent substructures.
- **MUSE:** Focuses on mining frequent patterns in uncertain graph databases using expected support.

Advantages:

- Scalable to massive datasets.
- Suitable for dynamic and uncertain graphs (e.g., social or biological networks).

Limitation:

- Accuracy may slightly degrade due to approximation.

5. Parallel and Distributed Methods

For very large graph datasets, traditional single-machine algorithms are insufficient.

Parallel algorithms decompose the dataset and process it using multiple computing nodes simultaneously.

Examples:

- **MR-FSG (MapReduce-based FSG):**
A distributed version of the Apriori-based FSG algorithm implemented using the MapReduce framework.
Each mapper finds local frequent subgraphs, and reducers merge the results.
- **gSpan on Spark (GraphX):**
An adaptation of gSpan for distributed graph processing on Apache Spark.

Advantages:

- Handles large-scale graph data efficiently.
- Achieves scalability and parallel computation.

Challenges:

- Communication overhead between nodes.
- Complexity in merging subgraph results.

Social Network Analysis: Characteristics of Social Networks

Social Network Analysis (SNA) is a methodological framework used to study relationships and interactions among individuals, organizations, or entities represented as **nodes** in a **network graph**.

In a social network, nodes denote actors (such as people, groups, or institutions), and edges denote relationships (such as friendship, communication, collaboration, or influence).

Formally, a social network is represented as:

where V is the set of vertices (actors), and E is the set of edges (relationships).

SNA investigates not just the existence of connections, but also their **patterns, intensity, and structure**, allowing researchers to understand how information, behaviors, and influence spread through the network.

Understanding the **characteristics of social networks** helps in identifying key players, community structures, and overall network dynamics.

2. Key Characteristics of Social Networks

The structure and dynamics of a social network can be analyzed using a set of quantitative and qualitative characteristics. The major characteristics include **size, density, degree, centrality, clustering, path length, reciprocity, and homophily**. Each characteristic provides a unique insight into the network's behavior and structure.

2.1 Size

The **size** of a network refers to the **number of nodes ($|V|$)** and **number of edges ($|E|$)** present in it.

- A **larger network** can represent a wider range of interactions or relationships.
- **Network size** directly affects the complexity and scalability of network analysis.

Mathematically:

Example:

A social media network with 10,000 users (nodes) and 100,000 friendships (edges) has a large size and a complex relationship structure.

2.2 Density

Density measures how closely the network's nodes are connected to each other. It is the ratio of the number of actual connections to the total possible connections.

For an undirected network:

For a directed network:

Interpretation:

- : The network is **fully connected** (every node is linked to every other node).
- : There are **no connections** between nodes.
- A **higher density** indicates more cohesive and tightly knit networks.

Example:

In a corporate network, a higher density among employees may indicate strong collaboration and information sharing.

2.3 Degree and Degree Distribution

The **degree** of a node represents the number of direct connections it has with other nodes.

- In **undirected networks**:
- In **directed networks**:
Each node has an **in-degree** (number of incoming edges) and an **out-degree** (number of outgoing edges).

Degree distribution describes how node degrees are distributed across the network.

It often follows a **power-law distribution** in real-world networks (known as **scale-free networks**), where a few nodes (called **hubs**) have very high degrees while most nodes have low degrees.

Example:

In a social media network, influencers represent nodes with high degrees due to many followers.

2.4 Centrality

Centrality measures the relative importance or influence of a node within the network.

Several types of centrality exist:

1. **Degree Centrality:**

Based on the number of direct connections a node has.

A higher degree centrality implies that the node is highly active or influential.

2. Closeness Centrality:

Measures how close a node is to all others in the network.

where $d(i, j)$ is the shortest path between nodes i and j .

Nodes with high closeness can spread information efficiently.

3. Betweenness Centrality:

Reflects how often a node acts as a bridge on the shortest path between other nodes.

where $\sigma(i, j)$ is the total number of shortest paths from i to j , and $\sigma(i, j, k)$ is the number of those paths passing through k .

4. Eigenvector Centrality:

Assigns relative scores to nodes based on their connections to other high-scoring nodes.

where N_i is the set of neighbors of i and λ is a constant.

Example:

In a company communication network:

- Degree centrality identifies well-connected employees.
- Betweenness centrality identifies individuals who act as intermediaries between departments.
- Eigenvector centrality identifies influential leaders connected to other important individuals.

2.5 Clustering Coefficient

The **clustering coefficient** indicates the tendency of nodes to form tightly connected groups or clusters.

For a node i :

The **average clustering coefficient** of the entire network is:

Interpretation:

- A **high clustering coefficient** suggests that friends of a person are likely to be friends with each other.
- A **low clustering coefficient** indicates a more dispersed or random structure.

Example:

In social networks like Facebook, users often belong to tightly knit groups (e.g., family, colleagues, classmates) leading to a high clustering coefficient.

2.6 Path Length and Diameter

- **Path Length:** The number of edges in the shortest path between two nodes.

The **average path length (L)** is given by:

- **Diameter:** The **longest shortest path** between any two nodes in the network.

Interpretation:

- Networks with **short average path lengths** exhibit the **small-world phenomenon**, meaning most nodes can be reached from others through a small number of steps.

Example:

Social media networks often show small-world behavior — “six degrees of separation.”

2.7 Reciprocity

In directed networks, **reciprocity** measures the degree to which relationships are mutual.

A higher reciprocity value means mutual interactions are common.

Example:

On Twitter, if two users follow each other, it contributes to reciprocity; one-way follows decrease it.

2.8 Homophily

Homophily is the tendency of individuals to associate and bond with similar others.

This similarity could be based on attributes like gender, age, profession, interests, or ideology.

Mathematically:

Homophily can be measured using an **assortativity coefficient (r)**:

where a_i are node attributes.

Interpretation:

- : Positive assortativity (similar nodes tend to connect).
- : Negative assortativity (dissimilar nodes tend to connect).

Example:

In professional networks like LinkedIn, users often connect with others in similar industries or positions, demonstrating high homophily.

2.9 Communities and Modularity

Social networks often exhibit **community structures**, where nodes cluster into groups with dense internal connections and sparse external links.

Modularity (Q) quantifies the strength of these community divisions:

where:

- A = adjacency matrix,
- k_i = degrees of nodes and k is the average degree,
- δ_{ij} is the Kronecker delta,
- c_i and c_j are the community labels of nodes i and j , respectively, where $c_i = c_j$ if nodes i and j belong to the same community.

High modularity indicates strong community formation.

Spatial Data Mining

Spatial Data Mining (SDM) is the process of discovering interesting, useful, and non-trivial patterns from spatial datasets. Unlike traditional data mining, which deals with data that can be represented as tables or records, spatial data mining focuses on data where the location, shape, and spatial relationships among objects are important. It extends traditional data mining techniques by incorporating spatial relationships and spatial attributes into the analysis.

1. What is Spatial Data?

Spatial data refers to data that has a spatial or geographical component. It describes the location, shape, and size of geographic objects and the relationships between them.

Examples:

- Locations of cities, rivers, or mountains.
- Distribution of soil types or vegetation.
- Traffic flow data in a city.
- Satellite imagery data.

Spatial data is typically stored in **Geographic Information Systems (GIS)**, **spatial databases**, or **remote sensing** systems.

2. Why Spatial Data Mining?

Spatial data mining is essential because:

- Traditional data mining algorithms assume that data samples are **independent and identically distributed (i.i.d.)**, but spatial data is **spatially autocorrelated** — nearby locations often have similar attributes (Tobler's First Law of Geography).
- The amount of spatial data is growing rapidly (e.g., from GPS, satellites, sensors), making manual analysis impossible.
- It helps in understanding **spatial patterns, trends, and relationships** that can support decision-making in areas such as urban planning, environment management, agriculture, and transportation.

3. Types of Spatial Data Mining Tasks

(a) Spatial Characterization and Generalization

- Summarizes general features of spatial data.
- Example: “Urban areas near rivers have higher population density.”
- Combines spatial and non-spatial attributes to produce descriptive rules.

(b) Spatial Classification

- Assigns spatial objects to predefined categories based on their attributes and spatial relationships.
- Example: Classifying land into urban, forest, or agricultural zones based on satellite imagery.

(c) Spatial Clustering

- Groups spatial objects into clusters such that objects within the same cluster are geographically close and similar in attributes.
- Example: Identifying high-crime zones or disease outbreak clusters.
- Algorithms: DBSCAN, ST-DBSCAN, DENCLUE, CLARANS.

(d) Spatial Association Rule Mining

- Finds rules showing relationships between spatial and non-spatial features.
- Example: “If a region has high rainfall and low elevation, then it is likely to be forested.”
- These rules help understand correlations among geographical phenomena.

(e) Spatial Trend Detection

- Detects patterns that change with spatial location.
- Example: “Average temperature decreases with increasing altitude.”
- Useful for environmental monitoring and climate modeling.

(f) Spatial Outlier Detection

- Identifies spatial objects whose attribute values differ significantly from their neighbors.
- Example: A region with unusually high pollution levels compared to surrounding areas.

4. Spatial Relationships

Spatial data mining focuses not just on data values but also on **spatial relationships** such as:

- **Topological relationships:** adjacency, overlap, containment (e.g., a park *contains* a lake).
- **Distance relationships:** proximity or nearest neighbor (e.g., “nearest hospital”).
- **Directional relationships:** north, south, east, west.

These relationships are key for analyzing how spatial entities influence one another.

Multimedia Data Mining

Multimedia Data Mining (MDM) refers to the process of extracting meaningful patterns, correlations, and knowledge from multimedia data such as **text, images, audio, video, and combinations of these types**.

Unlike traditional data mining, which deals mainly with structured numerical or categorical data, multimedia data mining handles **unstructured, semi-structured, and high-dimensional** data with rich content and complex relationships.

1. What is Multimedia Data?

Multimedia data integrates multiple forms of information — visual, auditory, and textual — to represent real-world phenomena.

Types of Multimedia Data:

Type	Description	Example
Text	Unstructured or semi-structured written content	News articles, captions, transcripts
Image	2D visual data representing scenes or objects	Photographs, X-rays, satellite images
Audio	Sound or speech signals	Music, podcasts, voice recordings

Video	Combination of moving images and sound	Movies, surveillance footage
Graphics/Animation	Drawings or simulations	CAD models, 3D animations

Multimedia data is typically stored in large databases, content repositories, and online platforms such as YouTube, Instagram, or digital libraries.

2. Need for Multimedia Data Mining

The exponential growth of multimedia content on the internet and digital platforms has created a huge need for automated techniques to analyze, categorize, and understand this data.

Key reasons include:

- Manual analysis of multimedia data is **impractical** due to volume and complexity.
- Multimedia data holds **rich contextual information** valuable for decision-making.
- Useful for **recommendation systems, surveillance, medical diagnostics**, and **digital marketing**.
- Helps bridge the **semantic gap** between low-level features (e.g., color, texture, sound frequency) and high-level concepts (e.g., “happy scene”, “angry tone”, “traffic congestion”).

Steps in Multimedia Data Mining Process

(a) Data Collection

Data is gathered from multiple sources such as sensors, cameras, social media platforms, or medical imaging systems.

(b) Data Preprocessing

Prepares multimedia data for analysis by:

- Noise reduction
- Data normalization
- Segmentation (image/video frame segmentation)
- Synchronization (aligning audio and video)

- Conversion into uniform formats

(c) Feature Extraction

Extracts **low-level** (e.g., color histograms, edges, textures) and **high-level** (e.g., objects, emotions, speech content) features from multimedia data.

Example features:

- **Images:** color moments, texture, SIFT, HOG.
- **Audio:** pitch, frequency, Mel-Frequency Cepstral Coefficients (MFCCs).
- **Video:** motion vectors, frame rate, keyframe detection.

(d) Pattern Discovery

Applies data mining algorithms to find hidden patterns or relationships:

- **Classification:** Identify object types (e.g., cat, dog, human).
- **Clustering:** Group similar multimedia objects (e.g., similar movie genres).
- **Association Rule Mining:** Find co-occurrence patterns (e.g., scenes with “beach” often have “blue sky” and “ocean sounds”).
- **Content-Based Retrieval:** Retrieve images or videos based on visual similarity (used in Google Images).

(e) Knowledge Representation

Visualizes discovered knowledge using dashboards, graphs, or summaries for easy interpretation.

5. Techniques Used in Multimedia Data Mining

Technique	Description	Example Application
Feature Extraction and Transformation	Converts raw multimedia data into meaningful numerical features	CNNs for image features
Dimensionality Reduction	Removes redundant features using PCA, t-SNE, or Autoencoders	Reduces video data size
Clustering	Groups similar images,	Grouping songs by

		sounds, or videos	rhythm
Classification		Categorizes multimedia data into predefined labels	Emotion detection from facial images
Association Mining	Rule	Discovers relationships among multimedia features	Correlating color tone with movie genre
Temporal Mining	Pattern	Extracts patterns from time-sequenced data like videos or audio	Detecting activity in surveillance footage
Deep Methods	Learning	Learns hierarchical representations directly from raw multimedia	CNNs, RNNs, GANs, Transformers

6. Applications of Multimedia Data Mining

Domain	Applications
Healthcare	Disease detection from MRI/CT images, emotion analysis from voice
Security and Surveillance	Motion detection, face recognition, anomaly detection
Entertainment	Movie recommendation, genre classification, music suggestion
Education	Intelligent tutoring systems using voice and video analysis
E-commerce	Product image search, personalized recommendations
Social Media Analytics	Detecting trends, analyzing sentiments in posts and videos
Geospatial and Remote Sensing	Land cover mapping, object identification in satellite imagery